2. Сравнение инструментов обработки данных - Pandas, Dask и Spark

Цель лабораторной работы

Научиться:

1. Оценивать производительность различных инструментов обработки данных (Pandas, Dask, Spark)
2. Выявлять и устранять узкие места в коде
3. Применять оптимизации Spark (Catalyst, Tungsten, настройка параметров)
4. Принимать обоснованные решения о выборе инструмента под конкретные требования

Система оценивания (максимум 10 баллов)

Критерий	Баллы	Описание
Сравнение производительности	2.0	Корректное измерение времени и потребления ресурсов
Анализ плана выполнения	2.0	Понимание физического плана Spark, выявление узких мест
Устранение skew	1.5	Обнаружение и решение проблемы skew
Оптимизация Spark	1.5	Применение конкретных оптимизаций с измерением эффекта
Обоснование выбора инструмента	1.5	Аргументированный выбор под конкретные требования
Отчетность	1.5	Четкость изложения, визуализация, структура

Задание

Инструкция

1. Выберите **один** датасет из ЛР-1 (ваш вариант определяется по последней цифре вашей зачетной книжки)
2. Выполните **одну** из задач обработки данных (см. ниже)
3. Сравните производительность Pandas, Dask и Spark для этой задачи
4. Проанализируйте физический план выполнения в Spark
5. Выявите и устраните проблемы с skew (если они есть)
6. Предложите оптимизации для Spark и измерьте их эффект
7. Сформулируйте рекомендации по выбору инструмента
8. Оформите результаты в Jupyter Notebook

Задачи обработки данных (выберите одну)

1. **Агрегация с группировкой**
   - Выполнить агрегацию по сложному ключу (2-3 поля) с несколькими агрегатными функциями
   - Пример: для NYC Taxi Trips - агрегация по району и часу с подсчетом количества поездок и средней стоимости
2. **Соединение таблиц**
   - Выполнить соединение двух таблиц по ключу с фильтрацией
   - Пример: для MovieLens - соединение ratings и movies с фильтрацией по жанру
3. **Оконные функции**
   - Реализовать расчет скользящего среднего или ранжирование с оконными функциями
   - Пример: для Airline Delays - расчет задержки относительно средней по маршруту за последние 7 дней
4. **Обработка неструктурированных данных**
   - Выполнить извлечение информации из текстовых/геоданных
   - Пример: для Twitter Data - извлечение хештегов и подсчет их частоты

Подробное задание

Часть 1: Подготовка окружения и данных

1. Установите необходимые библиотеки:
   ```
   pip install pandas dask pyspark numpy matplotlib great_expectations
   ```
2. Загрузите датасет и выполните первичную очистку:
   - Удалите дубликаты
   - Обработайте пропуски
   - Проверьте качество данных с помощью Great Expectations
3. Разделите данные на части для тестирования:
   - 100 МБ (для локального тестирования)
   - 1 ГБ (для сравнения Pandas/Dask)
   - Полный датасет (для Spark)
4. Настройте локальный кластер Spark:
   ```
   from pyspark.sql import SparkSession

   spark = SparkSession.builder \
       .appName("BigDataLab2") \
       .config("spark.sql.shuffle.partitions", "200") \
       .config("spark.default.parallelism", "200") \
       .getOrCreate()
   ```

Требования к отчету:

- Статистика по данным после очистки
- Результаты проверки данных через Great Expectations
- Конфигурация Spark-кластера

Часть 2: Сравнение производительности

1. Реализуйте выбранную задачу обработки данных на:
   - Pandas (на 100 МБ и 1 ГБ данных)
   - Dask (на 1 ГБ данных)
   - PySpark (на полном датасете)
2. Измерьте для каждого инструмента:
   - Время выполнения задачи
   - Потребление памяти (RAM)
   - Количество операций ввода-вывода
   - Использование CPU
3. Постройте сравнительные графики:
   - Время выполнения vs. объем данных
   - Потребление RAM vs. объем данных
4. Проанализируйте результаты:
   - При каком объеме данных Pandas перестает справляться?
   - На сколько процентов Dask быстрее Pandas на 1 ГБ данных?
   - Как соотносится время выполнения Spark с другими инструментами?

Требования к отчету:

- Таблица сравнения метрик для всех инструментов
- Графики зависимости времени выполнения от объема данных
- Анализ причин различий в производительности
- Вывод о том, при каких условиях каждый инструмент предпочтителен

Часть 3: Анализ физического плана выполнения в Spark

1. Для вашей Spark-реализации задачи:

```
# Пример для агрегации
result = df.groupBy("region", "hour").agg(
    F.count("*").alias("trips"),
    F.avg("fare_amount").alias("avg_fare")
)
result.explain()
```

2. Проанализируйте физический план:
   - Сколько этапов выполнения (stages)?

- Какие операции выполняются в каждом этапе?
- Есть ли операции shuffle? Если да, то сколько данных передается?

3. Выявите потенциальные узкие места:
   - Сколько данных обрабатывается на каждом этапе?
   - Есть ли дисбаланс нагрузки между узлами (skew)?
   - Какие операции занимают больше всего времени?
4. Предложите гипотезы об оптимизации на основе анализа плана

Требования к отчету:

- Вывод физического плана выполнения
- Анализ каждого этапа выполнения
- Выявление потенциальных узких мест
- Гипотезы об оптимизации

Часть 4: Выявление и устранение skew

1. Проверьте наличие skew в вашей задаче:

```
# Анализ распределения ключей группировки
key_distribution = df.groupBy("region").count().orderBy(F.desc("count"))
key_distribution.show()

# Визуализация распределения
import matplotlib.pyplot as plt
keys = [row["region"] for row in key_distribution.limit(20).collect()]
counts = [row["count"] for row in key_distribution.limit(20).collect()]
plt.bar(keys, counts)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

2. Если skew обнаружен:
   - Определите степень skew (коэффициент дисперсии)
   - Оцените влияние skew на производительность
3. Реализуйте одну из стратегий устранения skew:
   - Добавление "соли" к ключу группировки
   - Использование broadcast join для маленьких таблиц
   - Фильтрация "горячих" ключей
4. Измерьте эффект оптимизации:
   - Сравните время выполнения до и после оптимизации
   - Проанализируйте изменения в физическом плане

Требования к отчету:

- График распределения ключей группировки
- Коэффициент skew и его интерпретация
- Реализованная стратегия устранения skew
- Измерение эффекта оптимизации с цифрами

Часть 5: Оптимизация Spark-кода

1. Примените минимум 3 оптимизации:
   - Настройка `spark.sql.shuffle.partitions`
   - Использование `repartition()` перед группировкой
   - Включение adaptive query execution
   - Оптимизация формата хранения (Parquet вместо CSV)
   - Другие специфичные для вашего сценария оптимизации
2. Измерьте эффект каждой оптимизации отдельно:

```
# Пример измерения эффекта одной оптимизации
spark.conf.set("spark.sql.shuffle.partitions", "200")
start_time = time.time()
result = perform_aggregation()
time_200 = time.time() - start_time

spark.conf.set("spark.sql.shuffle.partitions", "400")
start_time = time.time()
result = perform_aggregation()
time_400 = time.time() - start_time

print(f"Ускорение: {time_200/time_400:.2f}x")
```

3. Постройте график "затраты vs. выгода" для каждой оптимизации
4. Определите оптимальную комбинацию параметров для вашей задачи

Требования к отчету:

- Таблица с результатами каждой оптимизации
- График "затраты vs. выгода"
- Рекомендации по оптимальной конфигурации для вашего сценария
- Обоснование выбора оптимальной комбинации параметров

Часть 6: GPU vs CPU

Цель части: измерить, что даёт смена вычислительного бэкенда **без изменения логики** обработки, и понять, когда GPU не окупается.

> **Трек A выполняется всеми и по умолчанию.** Треки B и C — опциональны: B только при наличии NVIDIA GPU, C — фолбэк, если GPU нет. **Оценивание одинаково для всех треков, баллы те же.**

Трек A (по умолчанию, локально на CPU): Polars/DuckDB против pandas

Студент без GPU получает измеримый эффект на той же задаче. Сравниваем pandas (однопоточный) с Polars (многопоточный, Arrow) и DuckDB (векторизованный SQL).

1. Подготовьте окружение:

```
python -m venv .venv
source .venv/bin/activate
pip install pandas polars duckdb pyarrow memory_profiler
```

2. Выполните **одну и ту же** агрегацию тремя инструментами:

```
import time
import pandas as pd
import polars as pl

# pandas: однопоточно, объектная модель в памяти
start = time.perf_counter()
pdf = pd.read_parquet("sample_100mb.parquet")
pandas_result = pdf.groupby(["region", "hour"])["fare"].mean()
t_pandas = time.perf_counter() - start

# Polars: многопоточно + Arrow, ленивый план
start = time.perf_counter()
ldf = pl.read_parquet("sample_100mb.parquet")
polars_result = ldf.group_by(["region", "hour"]).agg(pl.col("fare").mean())
t_polars = time.perf_counter() - start

print(f"Pandas: {t_pandas:.1f} с | Polars: {t_polars:.1f} с | ускорение: {t_pandas / t_polars:.1f}x")
```

```
import time
import duckdb

# DuckDB: векторизованный SQL прямо по файлу, без загрузки в pandas
start = time.perf_counter()
duck_result = duckdb.sql("""
    SELECT region, hour, AVG(fare) AS avg_fare
    FROM 'sample_100mb.parquet'
    GROUP BY region, hour
""").df()
t_duck = time.perf_counter() - start
```

3. Замерьте для каждого инструмента **время** и **пиковое потребление RAM** (например, `memory_profiler` или `psutil`) на срезах 100 МБ и 1 ГБ.
4. Сведите результаты в таблицу:

Инструмент	100 МБ: время	100 МБ: RAM	1 ГБ: время	1 ГБ: RAM
pandas				
Polars				

Инструмент	100 МБ: время	100 МБ: RAM	1 ГБ: время	1 ГБ: RAM
DuckDB				

5. Сделайте вывод: на сколько процентов Polars/DuckDB быстрее pandas и **почему** (параллелизм, Arrow, векторизация). Конкретные числа зависят от числа ядер и диска — их нужно измерить, а не взять из головы.

Трек B (при наличии NVIDIA GPU): cuDF for Apache Spark

Если в машине есть NVIDIA GPU с драйвером R525+ и CUDA 12.9 или 13.1, тот же Spark-код можно выполнить с GPU-плагином **cuDF for Apache Spark** (ex-RAPIDS Accelerator for Apache Spark; репозиторий `NVIDIA/cudf-spark`, Maven-артефакты остались `rapids-4-spark`). Образ собирается по официальной инструкции: `https://nvidia.github.io/cudf-spark/docs/download.html`.

1. Соберите и запустите контейнер с доступом к GPU:

```
# Только при наличии NVIDIA GPU: на macOS этот трек недоступен.
docker build -t cudf-spark-lab -f gpu/Dockerfile .
docker run --gpus all -it --rm -v "$PWD":/workspace -w /workspace cudf-spark-lab bash
```

2. Запустите **тот же** `app.py`, что и на CPU. Меняется только конфигурация, а не код:

```
# Код НЕ меняется: включается плагин, и физический план уходит на GPU.
spark-submit \
  --conf spark.plugins=com.nvidia.spark.SQLPlugin \
  --conf spark.executor.resource.gpu.amount=1 \
  --conf spark.task.resource.gpu.amount=0.25 \
  app.py
# spark.executor.resource.gpu.amount=1 — жёстко 1 GPU на executor (больше нельзя).
# spark.task.resource.gpu.amount=0.25 — 4 задачи делят один GPU-executor (обычно 2–6).
```

3. Замерьте время того же запроса на CPU и с плагином. **Замер имеет смысл, только если запрос на CPU идёт примерно от 30 секунд.**
4. Честные ожидания (из официального FAQ продукта):

Метрика	Честное ожидание
Ускорение	3–7x, типично ~4x, до 100x в отдельных случаях
Точка окупаемости	инициализация executor/GPU ~2–3 с; выигрыш примерно от 30 с обработки на запрос
GPU на executor	ровно 1 (несколько GPU на executor не поддерживается)
Параллелизм	2–6 задач на GPU-executor
Что ускоряется	GROUP BY/join/sort высокой кардинальности, оконные, запись Parquet/ORC, чтение CSV, транскодинг
Что НЕ ускоряется	RDD API, GraphX, кастомные Dataset-типы, построчные Python UDF (fallback на CPU)

5. Если ускорения нет, зафиксируйте это как результат: маленькие данные, RDD/UDF-heavy код, GraphX или узкое место в I/O — GPU здесь не окупается.

Трек C (фолбэк): Colab/Kaggle

Если NVIDIA GPU нет, тот же сценарий выполняется в облаке. **На macOS cuDF for Apache Spark недоступен**: в Docker на macOS нет NVIDIA runtime (проверено). Порядок:

1. Colab/Kaggle → Runtime → Change runtime type → GPU (например, T4).
2. Тот же ноутбук и тот же код, что в треках A/B.
3. Те же замеры и тот же отчёт.

Баллы за фолбэк не снижаются — принцип курса: local-first, но фолбэк не меняет оценивание.

Ключевой вывод части: ускорение — свойство **неизменённого** кода. Тот же SQL/DataFrame-запрос выполняется быстрее на другом бэкенде без переписывания приложения; в этом и ценность cuDF for Apache Spark.

Требования к отчету:

- Таблица замеров трека A (время и RAM для pandas/Polars/DuckDB на 100 МБ и 1 ГБ)
- Вывод о том, когда GPU не окупается
- Для трека B: команды запуска, замер CPU vs GPU и сравнение с честными ожиданиями
- Для трека C: ссылка на ноутбук и подтверждение, что код не менялся

Часть 7: Рекомендации по выбору инструмента

1. На основе вашего анализа определите:
   - Для каких объемов данных предпочтителен каждый инструмент
   - Какие типы задач лучше обрабатывать с помощью каждого инструмента
   - Какие ограничения имеет каждый инструмент в вашем сценарии
2. Составьте матрицу выбора:

Критерий	Pandas	Dask	Spark	Рекомендация
Объем данных < 100 МБ	+++	++	+	Pandas
…	…	…	…	…
Объем данных > 50 ГБ, есть NVIDIA GPU	+	+	+++ (cuDF for Apache Spark)	Spark с GPU-плагином
Малые данные (< 30 с на запрос)	+++	++	+	GPU не окупается — остаётся CPU

> **Когда GPU не нужен**: маленькие данные (обработка < 30 с на запрос), RDD/UDF-heavy код, GraphX, узкое место в I/O без тяжёлых агрегаций, отсутствие NVIDIA. Тогда честный результат — CPU-трек (Polars/DuckDB), а не «неполная» работа.

3. Рассчитайте cost/performance для каждого инструмента:
   - Стоимость инфраструктуры
   - Время разработки
   - Поддержка в production
4. Сформулируйте рекомендации для вашего конкретного бизнес-сценария:
   - Какой инструмент выбрать и почему
   - Какие компромиссы придется принять

- Какие риски могут возникнуть

Требования к отчету:

- Матрица выбора инструмента
- Расчет cost/performance
- Конкретные рекомендации для вашего сценария
- Обоснование рекомендаций с цифрами

Требования к оформлению работы

1. Репозиторий на GitHub/GitLab

- Должен содержать:
  - Jupyter Notebook с анализом
  - Файл `requirements.txt` с зависимостями
  - Файл `README.md` с кратким описанием
  - Каталог `gpu/` с `README.md`: какой трек выполнен (A/B/C), команды запуска и таблица замеров
- Файлы с данными не должны быть в репозитории (только ссылки на источники)
- В `README.md` укажите:
  - Ваш вариант (последняя цифра зачетки)
  - Выбранную задачу обработки данных
  - Краткие итоги анализа

2. Jupyter Notebook должен

- Иметь четкую структуру по разделам задания
- Содержать комментарии к коду
- Включать визуализацию (графики, диаграммы)
- Завершаться краткими выводами по каждому разделу
- Быть воспроизводимым (должен запускаться с нуля)
- Включать автоматические проверки:

```
# Проверка 1: Pandas не должен обрабатывать >1 ГБ данных
try:
    pandas_result = process_with_pandas(full_dataset)
    print("## WARNING ## Pandas обработал полный датасет — возможно, ошибка")
except Exception as e:
    print(f"## OK ## Pandas не смог обработать полный датасет: {str(e)}")
```

3. Пример структуры Notebook

```
# Лабораторная работа 2: Сравнение инструментов обработки данных

## Введение
- Используемый датасет: [название]
- Выбранная задача: [название]
- Основные цели анализа

## Часть 1: Подготовка окружения и данных
### 1.1. Загрузка и очистка данных
[код, результаты, графики]
### 1.2. Проверка качества данных
[код, выводы]
... и т.д.

## Заключение
- Краткие итоги сравнения инструментов
- Рекомендации по выбору под конкретные условия
- Оценка потенциальных улучшений
```